

Exploring the imbalance among the Credit Card Fraud Dataset

André Silva

Faculty of Science of University of Porto

Porto, Portugal

andretiagosilva77@gmail.com

ABSTRACT

Credit card fraud detection poses significant challenges due to the highly imbalanced nature of the dataset, where fraudulent transactions represent a very small minority. This study evaluates the performance of various sampling techniques, in mitigating this imbalance to improve classification model performance. Utilizing the publicly available Credit Card Fraud Detection dataset from Kaggle, we compare these methods across multiple machine learning models, including logistic regression, decision trees, and gradient boosting classifiers. Key metrics such as precision, recall, F1-score, and ROC-AUC are employed to assess performance. Results demonstrate that while oversampling methods significantly enhance recall for fraudulent transactions, they exhibit trade-offs in terms of precision and computational efficiency. These findings provide insights into the practical applications of oversampling techniques in imbalanced dataset scenarios, particularly in the domain of fraud detection.

1 INTRODUCTION

The dataset chosen is the well-known Credit Card Fraud, that consists in a set of anonymized credit card transactions, that are either labeled as fraudulent or genuine. From the Kaggle description, the dataset contains transactions made by credit cards in September 2013 by European cardholders.[5] In fact, this dataset presents transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. It is also known that the dataset is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions. [5] The dataset contains only numerical input variables which are the result of a PCA transformation. However, due to confidentiality issues, no background about the data was provided, as the features were overall encrypted. Features V1, V2, ... V28 are the principal components obtained with PCA, so the only features which have not been transformed with PCA are 'Time' and 'Amount'. The 'Time' feature describes the seconds elapsed between each transaction and the first transaction in the dataset. The feature 'Amount' is the transaction Amount, that can be used for example-dependent cost-sensitive learning. Finally, the feature 'Class' is the target variable and it takes value 1 in case of fraud and 0 otherwise.

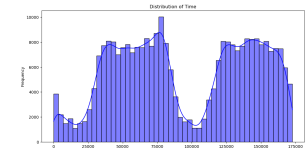
2 UNDERSTANDING THE DATA

Firstly, it was conducted a data-profiling. As previously stated, except for the transaction and amount, it is unknown what is the features meaning (due to privacy reasons). Effectively, it is important to be aware of the fact that, in order to implement a PCA transformation, features need to be previously scaled. In this case, all the V features have been scaled (or, at least, it can be presumed that such operation was done). Luckily, the dataset didn't have any

missing data, however, there were noticed some duplicates that ended up being dropped. Next, it was found the distribution of the revealed numeric features, Time and Amount. Although the 'Time' feature distribution seemed to be fine, the Amount distribution revealed an highly skewed attribute, with an heavy presence of outliers. Nonetheless, given that a Logistic Regression Classifier was going to be used for testing purposes later, both features were scaled properly.

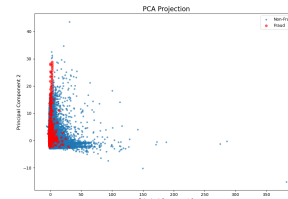


(a) Boxplot of Amount Distribution

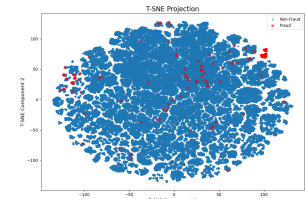


(b) Barplot of Transaction duration by increasing time

To conclude the data profiling, the distribution of the target variable, (that is, the 'Class') was verified. As already expected, the data imbalance is very high, with an approximately 99.84% presence of the non-positive class (non-fraud cases). To visualize such distribution, it was employed dimensionality reduction techniques, both linear and non-linear (PCA and T-SNE) to visualize the data, given that the data is multidimensional.



(a) Distribution Visualization using PCA



(b) Distribution Visualization of the Target Variable using T-SNE

Figure 2: Distribution of all data points identified by Class (blue for 0: Non Fraud and red for 1: Fraud).

3 RESULTS AND DISCUSSION

3.1 Splitting Data into training and test sets, and model selection

In order to test the performance of the classifiers over the dataset to further compare the impact of sampling methods on model performances, the dataset was split on a 70/30 proportion into a training

set and a test set. The split was stratified by the target variable to ensure that both the training and the test sets were following the same class proportions as the original dataset, which was done without any troubles. Then, the classifiers to be evaluated were chosen. First of all, it is important to mention that the classifiers chosen had to be computationally efficient, in order to see their performance in a reasonable time. Furthermore, given that there were going to be tested a range of different over-sampling methods, and given that the Imbalance Ratio was very high, it was expected that the dataset, with the synthetic data included, could become very extensive, which could make it complex to measure the models performance. It is also important to note that, for each classifier, the sampling techniques are applied during the training set cross-validation process, instead of being prior to it, so that synthetic data are created only for the training set without affecting the validation set [4]. In fact, this cross-validation happens with the implementation of GridSearch, that is a structure that can enhance a model performance by systematically exploring the combinations of the hyperparameter values to identify the optimal configuration that enhances the model performance. In fact, such structure comes in handy, given that the dataset is imbalanced, so, it was applied for each classifier.[7] The Decision Tree classifier was chosen, because Decision trees work well with imbalanced datasets by creating splits that maximize information gain (e.g., using Gini impurity or entropy). Effectively, they can focus on the minority class if the features are discriminative [2]. Moreover, fraud detection often involves complex relationships between variables. Decision trees can capture these non-linear patterns effectively. Moreover, the respective GridSearch defines the criterion of the classifier, that determines the function used to measure the quality of a split (with "gini" focusing on the Gini impurity and "entropy"), the max depth, that limits the maximum depth of the tree, controlling overfitting and model complexity, the minimum samples split, that specifies the minimum number of samples required to split an internal node, balancing the trade-off between depth and generalization, and the minimum samples leaf, that sets the minimum number of samples required to be a leaf node, preventing overly small leaves that may overfit the data.[6] Secondly, Logistic Regression was chosen because it also performs well with imbalanced data. By adjusting decision thresholds or incorporating oversampling techniques (as discussed later), Logistic Regression can effectively adapt to imbalanced datasets [9]. In a similar manner as the Decision Tree, that was also implemented a Grid Search, in this particular case, this structure configures a penalty, that specifies the type of regularization to apply, with 'l1' focusing on feature selection by enforcing sparsity and 'l2' reducing large coefficients to prevent overfitting, and explores a range of C-values that represents the inverse of regularization strength. As a matter of fact, smaller values of C apply stronger regularization, helping control overfitting, while larger values allow the model to fit the training data more closely. Finally, it configures a solver, to determines the algorithm used for optimization (that by default is the 'liblinear' solver), that is compatible with both penalty regimes 'l1' and 'l2'. [8]

3.2 The meaning of the performance metrics

In this particular context, it is important to find a balance between model precision and recall. In fact, the precision indicate the rate of transactions that are fraud over all the transactions that were labeled as fraud, as the recall tells, out of all actual fraud cases, the percentage of those who were correctly identified. A model with lower recall but higher precision is more selective. It tends to flag fewer transactions as fraudulent, which helps reduce false positives. However, this comes at the cost of potentially missing some actual fraud cases, leading to an increase in false negatives. In a real world scenario, these kind of models are better suited for situations where falsely labeling legitimate transactions as fraud can have significant negative consequences, such as customer dissatisfaction or financial losses from declined legitimate transactions.

Conversely, a model with higher recall and lower precision is more aggressive. It flags a larger number of transactions as fraudulent, which increases the chances of catching all fraudulent cases. However, this also means more legitimate transactions might be incorrectly flagged as fraud, leading to a higher rate of false positives. In a real world scenario, this type of model would be more appropriate if identifying frauds is critical, and the consequences of false positives are relatively less severe.

The ROC-AUC (Receiver Operating Characteristic - Area Under the Curve) is a crucial metric for evaluating binary classification models, particularly in tasks like fraud detection. It measures the model's ability to distinguish between classes (e.g., fraud vs. non-fraud) by assessing how well it ranks predictions based on their probabilities. Unlike threshold-dependent metrics, ROC-AUC evaluates performance across all thresholds, making it a comprehensive and fair measure. It balances the tradeoff between true positives and false positives, providing insights into the model's discriminative power. ROC-AUC is particularly valuable in imbalanced datasets, as it focuses on ranking predictions rather than absolute counts, making it less biased than metrics like accuracy. Thus, in this case, this metric is effectively more reliable than accuracy, given that accuracy is calculated on an unweighted manner, that is, this metric treats all predictions equally, meaning it does not account for the imbalance in class distribution. As the dataset is heavily imbalanced, the accuracy would reflect, mainly, the reliability of the prediction of the majority class. [1]

3.3 Model Performance Comparison across the Sampling methods considered

After obtaining baseline results, the models were re-evaluated using sampling techniques integrated into cross-validation. Random undersampling was the only undersampling method employed, as the imbalance is very expressive, and that would be a major information loss. Thus, it was expected that this method would affect negatively a model's precision. For oversampling, various techniques were applied, including Random Oversampling (ROS), Synthetic Minority Oversampling Technique (SMOTE) and its variants: Adaptive Synthetic Sampling (ADASYN), SMOTE for Nominal and Continuous Data (SMOTEN), and SVMSMOTE (a combination of SMOTE and SVM). Additionally, hybrid techniques combining over- and undersampling, such as SMOTEEN and SMOTE-Tomek, were also evaluated.[3]

It is important to notice that the exact metrics are available in the notebook that supports this report. A comparison of the results reveals that the Decision Tree model is generally more balanced, as it yields consistently the best F1-Scores. As a matter of fact, the F1-score of 0.90 in the baseline model highlights the Decision Tree's ability to handle class imbalance effectively. However, Logistic Regression also shows competitive performance, outperforming the Decision Tree in ROC-AUC for most sampling methods (except in the baseline and SMOTEN configuration).

The Decision Tree Classifier, while paired with the SMOTEN sampling method, in particular, provides the best precision across all cases. In a real-world scenario, this precision would be optimal for avoiding the false labeling of legitimate transactions as fraud. As mentioned before, falsely flagging legitimate transactions as fraudulent can have significant negative consequences, such as customer dissatisfaction or financial losses from declined legitimate transactions. However, it is also important to recognize that some fraudulent transactions may go unnoticed (false negatives), which could also be problematic (and also depends on the recall).

On the other hand, Logistic Regression consistently delivers better recall, particularly when oversampling methods are used. Despite sacrificing precision (with notably lower values while using oversampling approaches), this classifier may be more appropriate in scenarios where it is more important to identify as many fraudulent transactions as possible, even at the cost of increased false positives. In situations where missing fraud is more costly than flagging incorrectly legitimate transactions, Logistic Regression may be preferred.

4 CONCLUSIONS

The work carried out aimed not only to explore and address the class imbalance issue in the chosen dataset but also to identify the most suitable model (along with its optimal hyperparameters) for classifying transactions as fraudulent or non-fraudulent, while maintaining reasonable complexity. In addition, the most appropriate sampling method was sought. In fact, different real-world scenarios were considered, as the dataset reflects a practical, real-world problem. Fortunately, models with their respective optimal hyperparameters were identified, and when paired with various sampling methods, they proved suitable for each scenario. Finally, it is safe to say that the chosen models, along with a sampling technique, successfully balanced the precision-recall trade-off.

REFERENCES

- [1] Jason Brownlee. n.d.. ROC Curves and Precision-Recall Curves for Imbalanced Classification. <https://machinelearningmastery.com/roc-curves-and-precision-recall-curves-for-imbalanced-classification/>. Accessed: 2025-01-05.
- [2] GeeksforGeeks. 2024. How do decision trees work with unbalanced datasets? <https://www.geeksforgeeks.org/how-do-decision-trees-work-with-unbalanced-datasets/>
- [3] The imbalanced-learn contributors. n.d.. SMOTE — Version 0.13.0. https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html.
- [4] Janiobachmann. 2019. Credit Fraud || Dealing with Imbalanced Datasets. <https://www.kaggle.com/code/janiobachmann/credit-fraud-dealing-with-imbalanced-datasets/notebook#Test-Data-with-Logistic-Regression>
- [5] Kaggle. 2018. Credit Card Fraud Detection. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud> Accessed: [Insert Date of Access].
- [6] Scikit learn contributors. n.d.. DecisionTreeClassifier. <https://scikit-learn.org/dev/modules/generated/sklearn.tree.DecisionTreeClassifier.html>.
- [7] Scikit learn contributors. n.d.. GridSearchCV. https://scikit-learn.org/stable/modules/grid_search.html.
- [8] Scikit-learn. n.d.. LogisticRegression. https://scikit-learn.org/1.5/modules/generated/sklearn.linear_model.LogisticRegression.html.
- [9] Y. Sun, A. K. Wong, and M. S. Kamel. 2007. Logistic Regression for Class Imbalanced Datasets. *IEEE Transactions on Systems, Man, and Cybernetics* 37, 4 (2007), 914–922. <https://doi.org/10.1109/TSMCA.2006.885801>